

2차 과제 끝나고 이제 3차 과제를 할 때야, 3차 과제는 '데일리 플래너 메뉴 시스템'을 주제로 '값 매칭 분기:switch ~ case, 무한 반복문:while, 반복제어: break를 이용한 루프 탈출'을 쓰기로 했었지? 애네를 어떻게 활용하면 좋을까?



드디어 3차 과제 진입이구나! 1차에서 기초 뼈대를 잡고, 2차에서 디테일한 분석 로직을 짰다면, 3차 과제는 이 기능들을 하나로 묶어주는 '관제탑(메뉴 시스템)'을 만드는 단계야.

이제 프로그램이 한 번 계산하고 톱 꺼지는 게 아니라, 네가 진짜 앱을 쓰는 것처럼 계속 켜져서 명령을 기다리게 될 거야. 요구하신 3가지 필수 문법을 우리 벌크업 플래너에 어떻게 찰떡같이 적용할 수 있을지 기획안을 짜왔어!

💡 3차 과제 핵심 문법 매핑 기획안

1. 무한 반복문 (`while(1)`): 24시간 대기 모드

기존에는 코드가 위에서 아래로 한 번 실행되고 끝났지? 이제 전체 코드를 `while(1)` 이라는 거대한 주머니 안에 넣을 거야. 이렇게 하면 사용자가 "종료해!"라고 명령하기 전까지는 메뉴판을 계속 띄워주면서 대기하는 진짜 프로그램의 형태를 갖추게 돼.

2. 값 매칭 분기 (`switch ~ case`): 스마트 메뉴판

사용자에게 메뉴 번호(`1`, `2`, `0` 등)를 입력받아서, 그 번호에 맞는 기능으로 쓱쓱 보내주는 역할을 해.

- `case 1` : 1차 과제에서 만든 '신체 정보 및 목표 설정' 로직 실행
- `case 2` : 2차 과제에서 만든 '오늘의 식단/운동 분석' 로직 실행
- `default` : 사용자가 메뉴에 없는 이상한 숫자(예: `5`, `99`)를 입력했을 때 "잘못된 입력입니다"라고 알려주는 예외 처리

3. 반복 제어 (`break`): 루프 탈출 및 프로그램 종료

[🔥 초보자가 가장 많이 하는 실수 주의!]

C언어에서 `switch` 문 안에 있는 `break;` 는 `switch` 문만 빠져나가게 해줘. 즉, 무한 반복문인 `while(1)` 자체를 부수고 프로그램 밖으로(종료) 나가려면 약간의 트릭이 필요해.

그래서 `switch` 문에 들어가기 직전에 `if` 문을 써서 사용자가 '0번(종료)'을 입력했다면 `break;` 를 걸어 무한 루프를 탈출하도록 기획하는 것이 가장 깔끔해!



2st task

내가 지금 2차과제 까지 짜놓은 코드인데, 저기다가 switch case 와 무한 반복문과 반복제어를 어떻게 넣어줘야 할까?

💡 3차 과제 결합 포인트 (설계도)

- 1 변수는 그대로 상단 유지: 1번 메뉴에서 입력한 체중 데이터가 2번 메뉴에서도 유지되려면 변수들은 무조건 `while` 문 바깥쪽 가장 위에 있어야 합니다.
- 2 `while(1)` 로 전체 감싸기: 변수 선언이 끝난 직후부터 무한 반복문을 열고, 그 안에 메뉴판 출력과 `scanf` 를 넣습니다.
- 3 `switch(menu)` 로 분기 나누기: * `case 1:` 1차 과제 코드 (목표 체중, 칼로리 계산)
 - `case 2:` 2차 과제 코드 (오늘 섭취량 입력 및 조건문 분석)
 - `case 0:` 프로그램 종료 (`break` 를 이용해 `while` 루프 파괴)



너가 말한걸 참고해서 코딩을 했는데, 왜 메뉴번호에 0 을 입력했는데 메뉴가 종료되는것에서 끝나는게 아니라 다시 실행되는거야?



보내주신 코드 캡처 화면과 실행 결과창을 보니 원인을 딱 알겠습니다! 정말 다 잘 짜셨는데, 단어 하나가 빠져서 생긴 귀여운 해프닝이에요.

캡처해주신 `image_c002bf.png` 파일의 `if(menu == 0)` 부분을 보면 원인이 숨어있습니다.

- 1 사용자가 0 을 입력하면 `if(menu == 0)` 이 참이 되어서 "플래너를 종료합니다..."라는 문장을 정상적으로 출력합니다.
- 2 하지만 루프를 탈출하는 명령어인 `break;` 가 빠져있습니다!
- 3 그래서 프로그램은 종료되지 않고 바로 밑에 있는 `switch(menu)` 로 내려가게 됩니다.
- 4 `menu` 에는 0 이 들어있는데 `switch` 문 안에는 `case 1`, `case 2` 만 있고 `case 0` 은 없죠? 그래서 예외 처리 구문인 `default:` 로 넘어가서 "잘못된 번호입니다..."를 출력해 버린 것입니다.
- 5 결국 `while(1)` 무한 루프를 깨부수지 못했기 때문에 다시 처음으로 돌아가 메뉴판을 띄우게 된 것이죠.

해결 방법 (코드 수정)

해결 방법은 아주 간단합니다. `if` 문 안에서 문장을 출력한 직후에 `break;` (또는 프로그램 자체를 종료시키는 `return 0;`)를 딱 한 줄만 추가해 주시면 됩니다.



break; 를 추가시켰는데 에러가 떠



아하, 캡처해주신 화면 하단의 에러 메시지를 보니 원인이 명확합니다! `break;` 는 제 위치에 완벽하게 잘 넣으셨는데, 코딩 자체의 문제가 아니라 개발 환경(IDE) 쪽에서 아주 흔하게 발생하는 해프닝이에요.

하단 컴파일 로그를 보면 `cannot open output file ... Permission denied` 그리고 `[Error] ld returned 1 exit status` 라고 빨간 글씨로 적혀 있죠?

🔍 원인 분석 (왜 권한 거부 에러가 뜰까?)

이 에러는 코드가 틀려서 나는 문법 에러가 아닙니다. 운영체제가 "새로 수정한 코드로 프로그램을 덮어쓰기 (업데이트) 해야 하는데, 이전 버전의 프로그램이 지금 실행 중이라서 파일을 건드릴 수가 없어!" 라고 소리치는 상태입니다.

아까 무한 반복문을 빠져나오지 못해서 계속 켜져 있던 까만색 실행창(콘솔창)이 아직 컴퓨터 화면 어딘가(또는 작업 표시줄 백그라운드)에 켜져 있기 때문입니다.

🔧 해결 방법

- 1 기존 실행창 완전 종료: 작업 표시줄을 확인해서 켜져 있는 까만색 콘솔창(실행 결과 화면)을 모두 찾아서 강제로 꺼주세요. (창의 'X' 버튼 클릭)
- 2 다시 컴파일 및 실행: 실행창을 확실히 모두 닫았다면, Dev-C++에서 다시 '컴파일 후 실행(보통 단축키 `F11`)'을 눌러보세요.

C언어에서 무한 루프(`while(1)`)를 다루기 시작할 때 100명 중 99명이 겪는 통과 의례 같은 아주 자연스러운 현상입니다. 기존에 실행된 창만 닫아주시면 수정한 `break;` 가 찰떡같이 작동하면서 0번을 누르면 프로그램이 예쁘게 종료될 겁니다! 🚀

세부 요구 사항 (필수 포함)
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴
로그인/회원가입/로그아웃/비밀번호 찾기/회원탈퇴



2st task

코딩을 완성했는데, 교수님이 요구하신 세부 요구 사항(필수 포함)의 조건을 다 충족시켰을까?



지금까지 정말 완벽하게 잘 따라오셨어요! 이전 단계의 오류를 스스로 고치고 3차 과제(메뉴 시스템)까지 멋지게 완성한 건 정말 칭찬해 드리고 싶습니다.

하지만 냉정하게 말씀드리자면, 방금 올려주신 교수님의 '세부 요구 사항(필수 포함)' 표를 기준으로 보면, 현재 코드는 1번 조건만 100% 충족하고 나머지 2~5번 조건은 아직 하나도 충족하지 못한 상태입니다.

요구 사항 충족 여부 진단 리포트

평가 항목	충족 여부	현재 코드 상태 및 부족한 점
사용자 메뉴 구성	● 완벽	<code>while(1)</code> 과 <code>switch</code> , 그리고 0번 입력 시 <code>break</code> 탈출까지 완벽하게 구현되어 있습니다.
함수 정의 및 호출	● 미충족	현재 코드는 모든 입력, 연산, 출력이 <code>main()</code> 이라는 거대한 함수 하나에 다 몰려있습니다. 교수님 요구대로라면 <code>main</code> 안의 <code>case 1</code> , <code>case 2</code> 코드를 밖으로 빼서 3개 이상의 새로운 함수로 만들어야 합니다.
매개변수	● 미충족	새롭게 만들 함수 중에서 최소 1개는 괄호 <code>()</code> 안에 재료(인자)를 전달받아 계산하는 기능이 있어야 합니다.
반환값 (return)	● 미충족	현재는 프로그램 종료를 위한 <code>return 0;</code> 만 있습니다. 계산을 수행하고 그 결과를 <code>return</code> 으로 뱉어내는 함수가 최소 1개 필요합니다.
전역/지역 변수	● 미충족	현재 모든 변수가 <code>main()</code> 함수 안에 선언된 '지역 변수'입니다. 프로그램 전체에서 공유할 '전역 변수'를 <code>main</code> 함수 꼭대기 밖으로 빼내야 하고, 함수 안에서 이 전역 변수를 수정하는 로직이 필요합니다.



3st task

백준 10815 - 수들의 집

두 집의 학생이 두 집의 수를 비교할 때
비교할 수의 개수
비교할 수의 개수가 같을 때 두 집의 수를
비교할 수의 개수가 같을 때 두 집의 수를
비교할 수의 개수가 같을 때 두 집의 수를
비교할 수의 개수가 같을 때 두 집의 수를
비교할 수의 개수가 같을 때 두 집의 수를
비교할 수의 개수가 같을 때 두 집의 수를
비교할 수의 개수가 같을 때 두 집의 수를

너가 말한대로 3차과제들 완성 시켜봤는데 세부 요구 사항들 전부
충족 된거 같아?

코드 확인해 봤는데, 결론부터 말하자면 교수님이 제시한 5가지 세부 요구 사항을 100% 완벽하게 충족했어! 감동당할 건덕지가 전혀 없을 정도로 구조를 아주 깔끔하게 잘 짰네.

교수님 채점표(image_bfa40e.png) 항목별로 네 코드가 어떻게 요구사항을 저격했는지 조목조목 짚어줄게. 확신을 갖고 과제 제출해도 좋아!

세부 요구 사항 매칭 검증 결과

1. 사용자 메뉴 구성 (성공)

- 요구사항: `while(1)` 무한 루프 사용, 메인 메뉴 출력, '종료' 선택 시에만 `break` 로 탈출.
- 코드 반영 상태: * `main()` 함수 시작하자마자 `while(1)` 로 무한 루프 가동함.
 - 사용자가 0 번을 입력하면 `if(menu == 0)` 조건문 안에서 `break;` 가 작동하며 안전하게 탈출하도록 설계됨.

2. 함수 정의 및 호출 (성공)

- 요구사항: 최소 3개 이상의 개별 함수 분리, `main` 함수 내부는 목차처럼 간결하게 구성.
- 코드 반영 상태:
 - 총 5개의 개별 함수(`setPhase1Goal` , `calcDailyCalorie` , `calcProteinGoal` , `analyzeDiet` , `analyzeExercise`)를 정의해서 기준치(3개 이상)를 가볍게 넘김.
 - 복잡한 연산과 `if-else` 분석 로직이 함수 내부로 다 들어간 덕분에, `main` 함수는 메뉴판과 입력, 함수 호출만 남아서 정말 책의 '목차'처럼 한눈에 들어옴.

3. 매개변수 (성공)

- 요구사항: 최소 1개 이상의 함수가 매개변수(인자)를 전달받아 처리.
- 코드 반영 상태:
 - `calcDailyCalorie(current_weight)` 로 현재 체중을 던져주고 있고,
 - `analyzeDiet(today_calorie, today_protein)` 등 4개의 함수가 매개변수를 받아서 처리하므로 완벽하게 충족함.

4. 반환값 (성공)

- 요구사항: 최소 1개 이상의 함수가 연산 결과를 `return` 문으로 반환, 반환자 자료형 명시.
- 코드 반영 상태:
 - `int calcDailyCalorie(double weight)`
 - `int calcProteinGoal(double weight)`
 - 위 두 함수가 반환타입을 `int` 로 명시하고, 계산된 값을 `return` 을 통해 돌려주고 있음. 이를 받아 전역 변수에 대입하는 흐름까지 완벽함.

5. 전역/지역 변수 관리 (성공)

- 요구사항: 전역/지역 변수 역할 구분, 함수 내부에서 전역 변수 값 직접 변경(업데이트) 로직 포함.
- 코드 반영 상태:
 - 역할 분리: 전제 메뉴가 공유하는 목표 데이터(`initial` , `daily_calorie` 등)는 전역 변수로 두고, `main` 함수에서 매번 입력받는 오늘 데이터(`today_calorie` 등)는 지역 변수로 철저히 분리함.
 - 전역 변수 업데이트: `setPhase1Goal()` 함수 내부를 보면, 매개변수로 값을 받지 않았는데도 함수 안에서 전역 변수인 `weight_to_gain` 이나 `total_steps` 값을 직접 연산하고 변경(업데이트)하고 있음.

최종 점검 요약 (교수님 시선)

"지나치게 비대했던 메인 함수를 유기적인 데이터 흐름(전역/지역 변수, 매개변수, 반환값)을 활용해 모듈화(함수화)를 완벽히 수행한 A+짜리 코드"

1, 2차 과제에서 했던 조건문 스파게티 코드가 함수 단위로 예쁘게 포장되어서 가독성이 진짜 좋아졌어. 변수명도 축약형 안 쓰고 다 풀네임으로 맞춰놔서 클린 코드 관점에서도 백점 만점이야.

이제 이 코드로 컴파일해서 정상 작동하는 거 스크린샷 찍고, 보고서 채워서 제출만 하면 끝이야! 고생 많았어, 득근하자! 🍀